CAD for Digital H/W (E-CAD)

Αναφορά Εργασίας

ΣΤΥΛΙΑΝΗ ΤΣΙΜΗ 1084572

Περιγραφή Modules

Σκοπός της φετινής εργασίας εξαμήνου είναι η σχεδίαση ενός Συστήματος Εισαγωγής & Απεικόνισης Χαρακτήρων. Συγκεκριμένα, εισάγουμε δεδομένα (1,2,3,4,5,6,r,g.b) από το πληκτρολόγιο, τα οποία αποκωδικοποιούνται στο fpga και εμφανίζονται στην οθόνη. Με το πάτημα των πλήκτρων 1,2,3,4,5 και 6 επιλεγούμε παράθυρο, ενώ με τα r,g,b αλλάζουμε το χρώμα του επιλεγμένου παραθύρου. Στο κύκλωμα που θα συνθέσουμε χρησιμοποιούμε ένα κοινό ρολόι 25Mhz, το οποίο παράγεται από την διαίρεση του ρολόι του fpga (100Mhz) με 4 χρησιμοποιώντας counter, στο module pixel_clk.

Θέλουμε να αντλήσουμε τα scancodes από το πάτημα του κάθε χαρακτήρα. Για να γίνει αυτό, δειγματοληπτούμε τις τιμές του ps2data στο falling edge του ps2clk (του ρολογιού του πληκτρολογίου). Για να ανιχνεύσουμε το falling edge του ps2clk, αντί να το χρησιμοποιήσουμε σαν δεύτερο ρολόι, που είναι ανεπιθύμητη πρακτική, θα δειγματληπτήσουμε και το ps2clk, αναζητώντας πότε οι τελευταίες 8 τιμές του γίνονται h'F0. Τότε θα έχουμε falling edge. Όταν έχουμε ένα σωστό πακέτο, τροφοδοτούμε την έξοδο scancode με τα αντίστοιχα δεδομένα του ps2data. Ταυτόχρονα, στέλνουμε και ένα σήμα flag το οποίο υποδεικνύει ότι εισάγαμε έναν νέο χαρακτήρα. Η λογική αυτή υλοποιείται στο module kbd_protocol.

Στην συνέχεια, αποκωδικοποιούμε τα scancodes στο module scancode_decoder. Αποκωδικοποιούμε τα scancodes που λαμβάνουμε, μόνο αν έχουμε ενεργοποιημένη την είσοδο flag από το προηγούμενο module .Αν έχουμε εισάγει χαρακτήρα (1,2,3,4,5,6), τότε δίνουμε την ανάλογη τιμή στο reg window_select. Αυτό γίνεται για να κρατήσουμε ποιο παράθυρο επέλεξε ο χρήστης. Για τον χαρακτήρα escape δίνουμε στο window_select μια «ψεύτικη τιμή», ώστε να μην επιλεχθεί κανένα παράθυρο. Όταν πατάμε κάποιο από τα r,g,b κάνουμε το αντίστοιχο reg 1, ώστε να το χρησιμοποιήσουμε στην συνέχεια για τις αλλαγές των συνιστωσών των χρωμάτων. Τα σήματα αυτά μηδενίζονται σε κάθε νέο κύκλο του ρολογιού μας.

Το module vgasync υλοποιεί την λογική επικοινωνίας με την οθόνη. Χρησιμοποιώντας τους counters hcounter και vcounter και τις παραμέτρους που δόθηκαν, μπορούμε να παράξουμε τα σήματα οριζόντιου και κάθετου συγχρονισμού με την οθόνη h_sync και v_sync.

Στο module color_memory γίνεται ο χρωματισμός των pixel που θέλουμε να εμφανίσουμε στην οθόνη σε συγχρονισμό με το ρολόι. Δημιουργούμε μια μνήμη color_memory με 6 γραμμές (όσα και τα παράθυρα) και 3 στήλες (όσα και τα χρώματα). Λαμβάνοντας υπόψη τα regs r,g,b, καθώς και το window_select από το module scancode_decoder αυξομειώνουμε τις συνιστώσες των χρωμάτων. Επιλέγουμε τα κατάλληλα pixel για τον χρωματισμό του πλαισίου και του

περιγράμματος. Ορίζουμε τα pixel που αντιστοιχούν σε κάθε παράθυρο μέσω του vcounter και hcounter. Στις εξόδους red_out, green_out, blue_out δίνεται η κατάλληλη σειρά και στήλη της μνήμης. Ελέγχουμε το window_select και αναγνωρίζουμε το επιλεγμένο παράθυρο στο οποίο πρέπει να εμφανίζεται το περίγραμμα πάχους 5. Δίνουμε στα κατάλληλα pixel κίτρινο χρώμα. Στα υπόλοιπα pixel στέλνουμε μαύρο.

Περιγραφή Λειτουργίας

Αρχικά δημιουργούμε το ρολόι μας με συχνότητα λειτουργίας 25MHz. Αντλούμε τα scancodes από την επικοινωνία με το πληκτρολόγιο. Σε κάθε παλμό του ρολογιού μας, έχουμε σήμα για το αν έχουμε νέο scancode (δηλαδή, αν έχουμε πατήσει και αφήσει κάποιο πλήκτρο) καθώς και το scancode αυτό. Αν έχουμε καινούριο scancode, αυτό αποκωδικοποιείται, στέλνοντας τα σήματα αυτά στο color_memory.

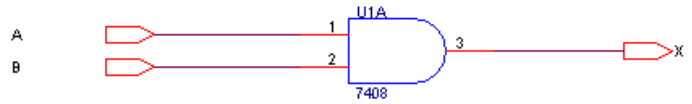
Σε κάθε παλμό του ρολογιού, ελέγχουμε αν έχουμε νέο scancode, και αν έχουμε τροφοδοτούμε κατάλληλα τα σήματα εξόδου r,g,b,window_select. Παράλληλα, δημιουργούμε τον συγχρονισμό με την οθόνη: Σε κάθε παλμό του ρολογιού, αυξάνεται ο h_counter μέχρι να φτάσει στην τιμή 800, όπου μηδενίζεται. Σε κάθε μηδενισμό του h_counter, αυξάνεται ο v_counter, μέχρι να φτάσει την τιμή 449 όπου και μηδενίζεται. Ανάλογα με την τιμή των σημάτων αυτών, στέλνουμε τα hsync και vsync στην οθόνη, συνυπολογίζοντας πάντα και τα porches.

Τέλος, τα δεδομένα περνάνε στο color_memory όπου καθορίζονται τα pixel του κάθε παραθύρου, καθώς και το χρώμα του.

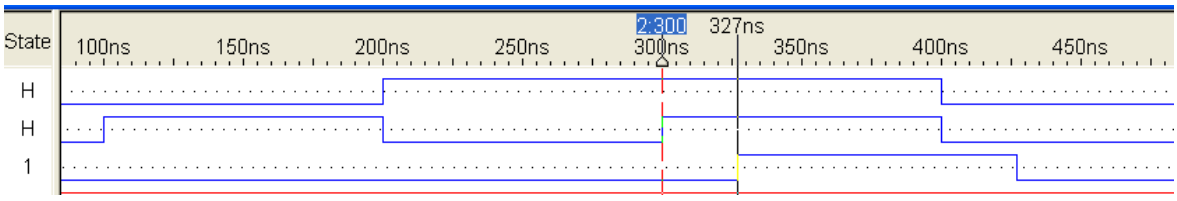
Εργασίες

Εργασία 1

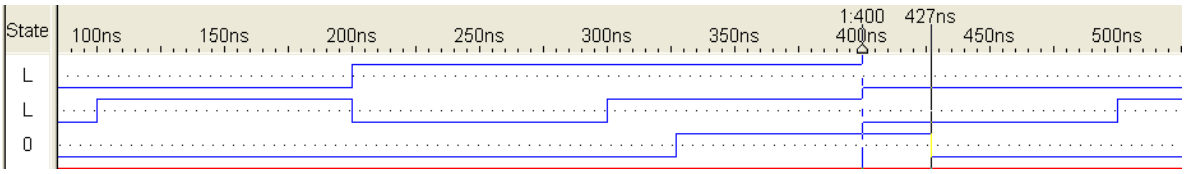
✓ Για την πύλη 7408:



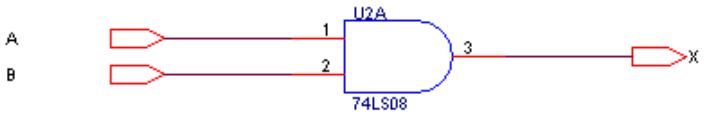
Η καθυστέρηση της πύλης όταν μεταβαίνει στο λογικό ένα είναι 27ns.



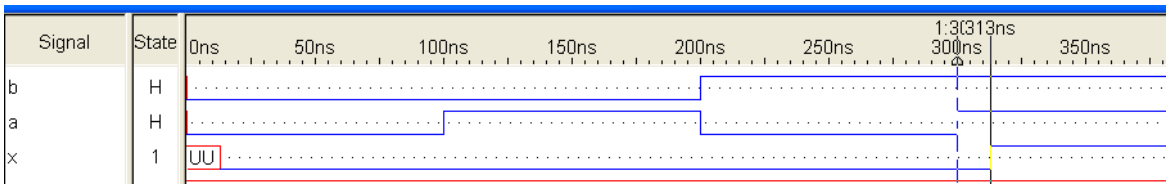
Η καθυστέρηση της πύλης όταν μεταβαίνει στο λογικό μηδέν είναι 27ns.



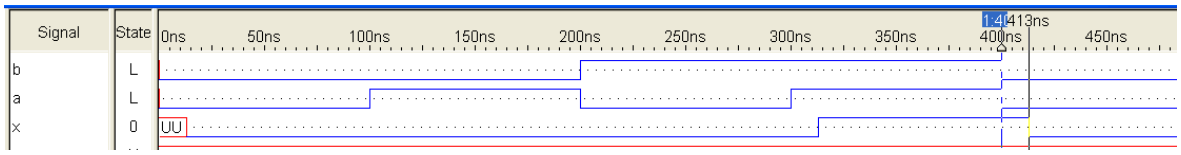
✓ Για την πύλη 74LS08:



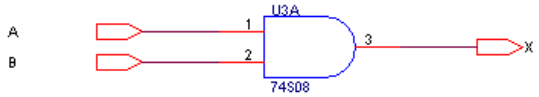
Η καθυστέρηση της πύλης όταν μεταβαίνει στο λογικό ένα είναι 13ns.



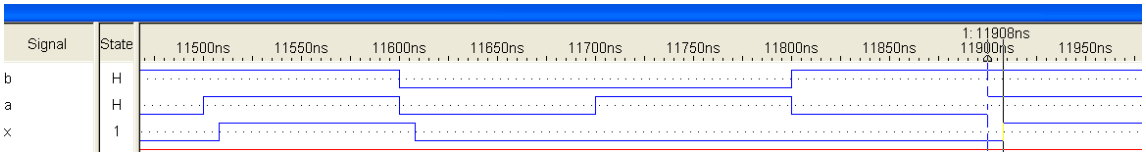
Η καθυστέρηση της πύλης όταν μεταβαίνει στο λογικό μηδέν είναι 13ns.



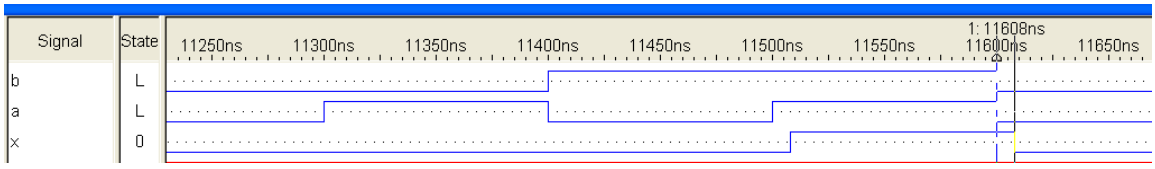
✓ Για την πύλη 74S08:



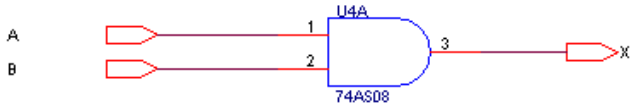
Η καθυστέρηση της πύλης όταν μεταβαίνει στο λογικό ένα είναι 8ns.



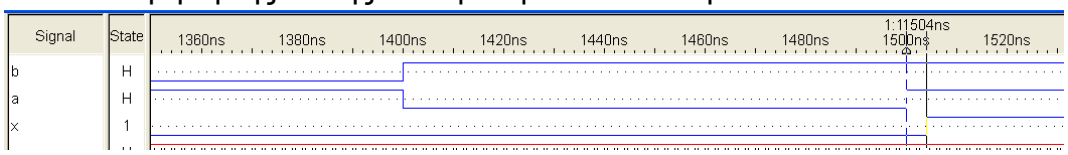
Η καθυστέρηση της πύλης όταν μεταβαίνει στο λογικό μηδέν είναι 8ns.



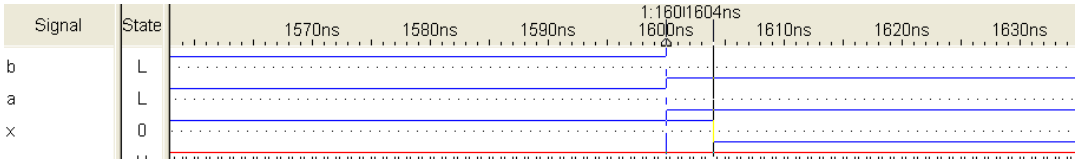
✓ Για την πύλη 74AS08:



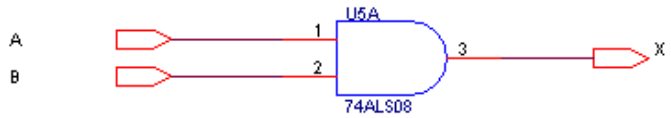
Η καθυστέρηση της πύλης όταν μεταβαίνει στο λογικό ένα είναι 4ns.



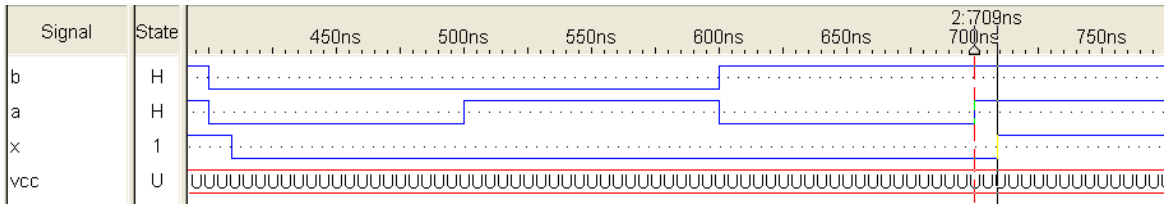
Η καθυστέρηση της πύλης όταν μεταβαίνει στο λογικό μηδέν είναι 4ns.



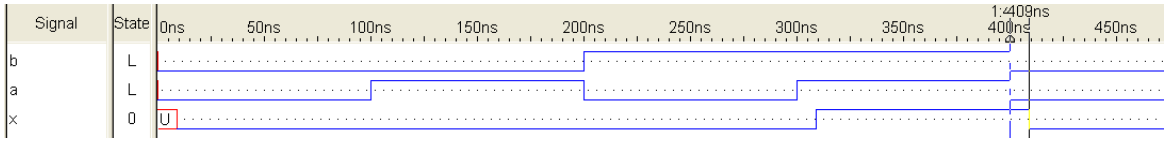
✓ Για την πύλη 74ALS08:



Η καθυστέρηση της πύλης όταν μεταβαίνει στο λογικό ένα είναι 9ns.



Η καθυστέρηση της πύλης όταν μεταβαίνει στο λογικό μηδέν είναι 9ns.



✓ Συμπερασματικά

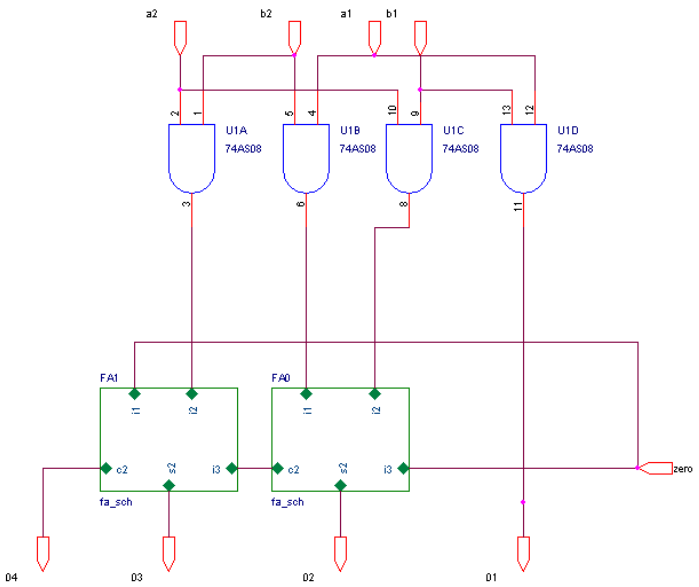
	7408	74ALS08	74AS08	74S08	74LS08
Χρόνος καθυστέρησης μετάβασης στο λογικό 0	27ns	13ns	8ns	4ns	9ns
Χρόνος καθυστέρησης μετάβασης στο λογικό1	27ns	13ns	8ns	4ns	9ns

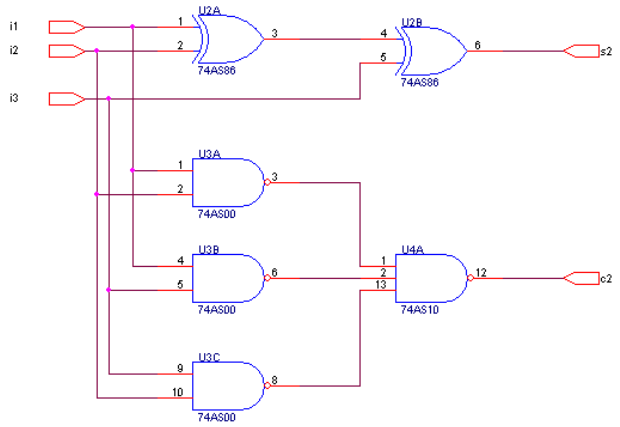
Οι διαφορετικές καθυστερήσεις διάδοσης οφείλονται και στις διαφορετικές τεχνολογίες κατασκευής τους.

- **7408**: Χρησιμοποιεί την κλασική τεχνολογία TTL (Transistor-Transistor Logic). Η καθυστέρηση είναι μεγαλύτερη λόγω της λιγότερο αποδοτικής διαχείρισης του ρεύματος και της χωρητικότητας.
- **74LS08**: Χρησιμοποιεί την τεχνολογία Low Power Schottky, η οποία μειώνει την κατανάλωση ενέργειας και βελτιώνει την ταχύτητα μέσω της χρήσης Schottky διόδων για την αποτροπή κορεσμού των τρανζίστορ.
- **74S08**: Βασίζεται σε Schottky τεχνολογία, που επιτρέπει ταχύτερη λειτουργία μέσω γρηγορότερης απόκρισης των τρανζίστορ λόγω της μειωμένης καθυστέρησης κορεσμού.
- **74AS08**: Advanced Schottky, είναι μια πιο εξελιγμένη έκδοση της Schottky τεχνολογίας, με ακόμα χαμηλότερη καθυστέρηση, βελτιστοποιημένη για μεγαλύτερη ταχύτητα.
- **74ALS08**: Advanced Low Power Schottky, συνδυάζει τα πλεονεκτήματα της μειωμένης κατανάλωσης ενέργειας με την ταχύτερη λειτουργία των Schottky διόδων, προσφέροντας καλύτερο λόγο ταχύτητας/κατανάλωσης.

Εργασία 2

Α. Οι σχεδιασμοί μας :





Για να επαληθεύσουμε την λειτουργία του σχηματικού μας, δημιουργούμε τον πίνακα αληθείας

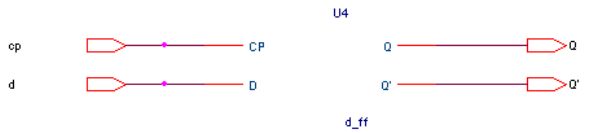
b2	b1	a1	a2	o4	o3	o2	o1
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	0
0	1	0	1	0	0	0	1
0	1	1	0	0	0	1	0
0	1	1	1	0	0	1	1
1	0	0	0	0	0	0	0
1	0	0	1	0	0	1	0
1	0	1	0	0	1	0	0
1	0	1	1	0	1	1	0
1	1	0	0	0	0	0	0
1	1	0	1	0	0	1	1
1	1	1	0	0	1	1	0
1	1	1	1	1	0	0	1

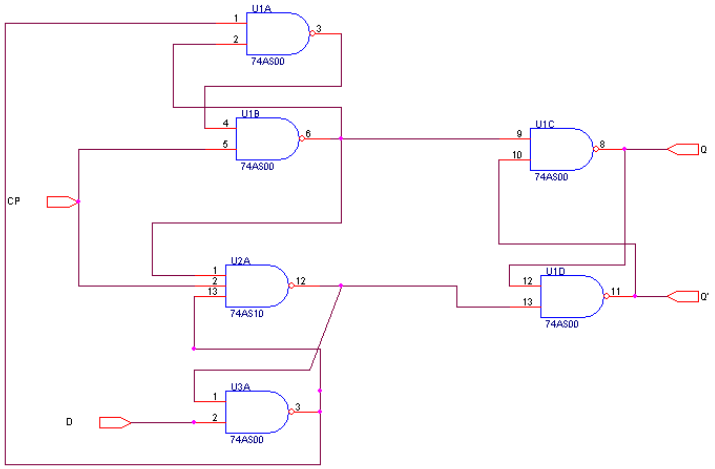
Για να ελέγξουμε την ορθότητα του σχεδιασμού μας δημιουργούμε ένα test bench. Το αρχείο multiplier.vhd περιέχει κι τα 16 test cases που θα μας αποκαλύψουν λάθη στον σχεδιασμό μας.

Παράδειγμα test case

```
-- Test case 1: a = 00, b = 00
a1 <= '0'; a2 <= '0'; b1 <= '0'; b2 <= '0'; zero <= '0';
wait for 10 ns;
assert (\01\ = '0' and \02\ = '0' and \03\ = '0' and \04\ = '0')
  report "Test Case 1 Failed" severity error;
```

B.





Το αρχείο stimulus:

```
schematic1.cp
 +  At 0, L for 50, H for 50, repeat forever
schematic1.d
 +  At 0, L for 100, H for 100, repeat forever
```

Τα αποτελέσματα της εξομοίωσης:

ns	q	q'	d	cp
0	U	U	L	L
50	U	U	L	H
56	U	1	L	H
59	0	1	L	H
100	0	1	H	L
150	0	1	H	H
156	1	1	H	H
159	1	0	H	H
200	1	0	L	L
250	1	0	L	H
256	1	1	L	H
259	0	1	L	H
300	0	1	H	L
350	0	1	H	H
356	1	1	H	H
359	1	0	H	H
400	1	0	L	L
450	1	0	L	H
456	1	1	L	H
459	0	1	L	H
500	0	1	H	L
550	0	1	H	H
556	1	1	H	H
559	1	0	H	H
600	1	0	L	L

Για αποδοτικότερο έλεγχο της λειτουργικότητας του καταχωρητή δημιουργούμε ένα test bench με το αρχείο test.vhd.

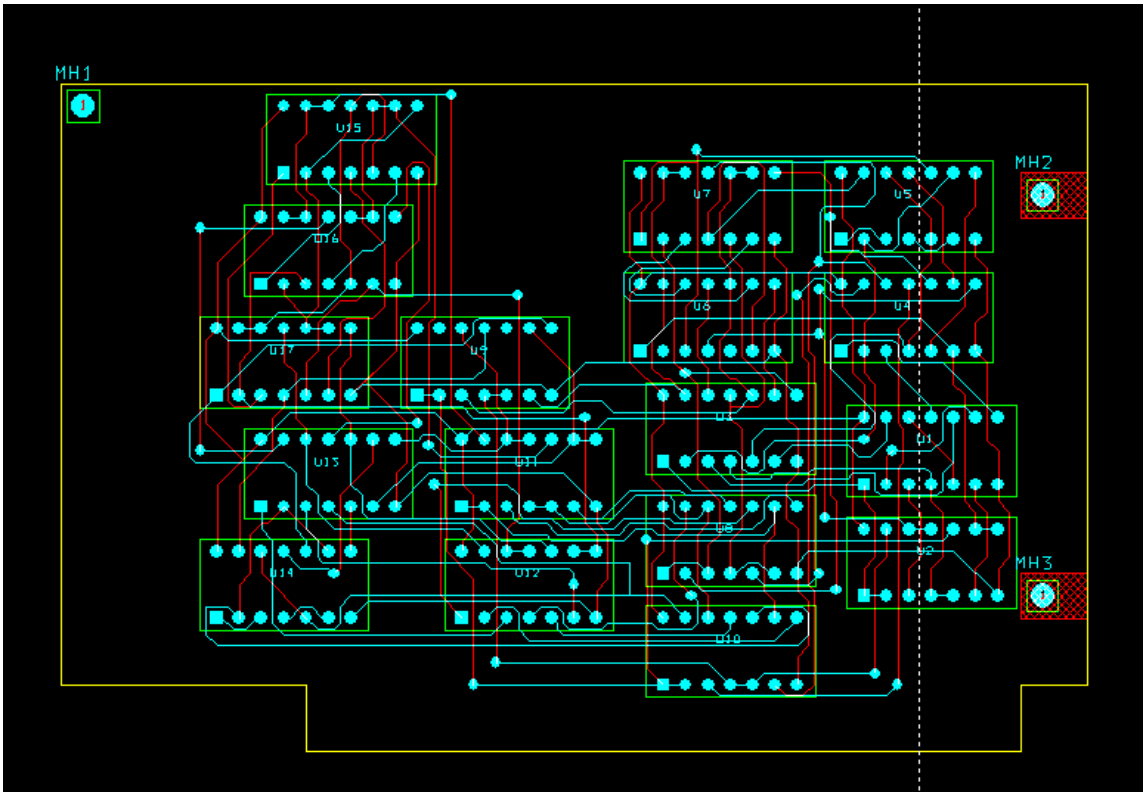
Παράδειγμα test case

```
-- Test case 3: Toggle d
    d <= '0';
    wait for 100 ns;
    d <= '1';
    wait for 100 ns;
```

Εργασία 3

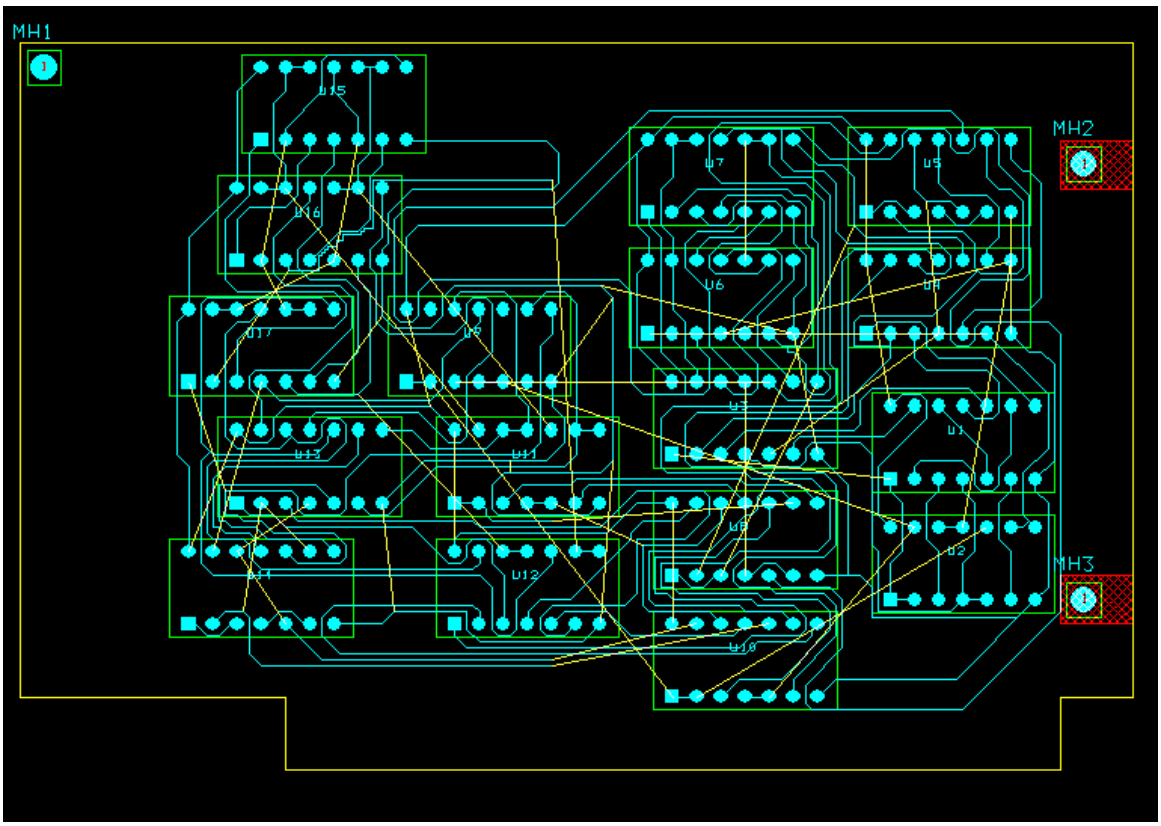
Παρακάτω δίνονται τα σχηματικά των πλακετών ενός και δύο επιπέδων

2 επίπεδα:



Sweep/Layer Name	Enabled	Cost	Direction	Between
Win/Comp/Manual				
TOP	Yes	50	80 Horz.	30
BOTTOM	Yes	50	20 Vert.	30
1 Preliminary Route				
TOP	Yes	50	80 Horz.	0
BOTTOM	Yes	50	20 Vert.	0
2 Maze Route				
TOP	Yes	50	80 Horz.	30
BOTTOM	Yes	50	20 Vert.	30
3 Next 1				
TOP	Yes	50	51 Horz.	0
BOTTOM	Yes	50	49 Vert.	0
4 Next 2				
TOP	Yes	50	51 Horz.	0
BOTTOM	Yes	50	49 Vert.	0
5 Next 3				
TOP	Yes	50	49 Vert.	0
BOTTOM	Yes	50	51 Horz.	0
6 Special Options				
TOP	Yes	50	51 Horz.	0
BOTTOM	Yes	50	49 Vert.	0

1 επίπεδο:



Sweep/Layer Name	Enabled	Cost	Direction	Between
Win/Comp/Manual				
TOP	Yes	50	80 Horz.	30
1 Preliminary Route				
TOP	Yes	50	80 Horz.	0
2 Maze Route				
TOP	Yes	50	80 Horz.	30
3 Next 1				
TOP	Yes	50	51 Horz.	0
4 Next 2				
TOP	Yes	50	51 Horz.	0
5 Next 3				
TOP	Yes	50	49 Vert.	0
6 Special Options				
TOP	Yes	50	51 Horz.	0

Παρατηρούμε ότι η διασύνδεση δεν είναι δυνατό να είναι πλήρης σε ένα επίπεδο.

Εργασία 5

1)Τροποποιούμε τον κώδικα ώστε να υλοποιήσουμε ένα μετρητή των 4^{ων} δυαδικών ψηφίων

```
module counter (clear, clock, load, start_stop, count, data);

  input [3:0] data;

  output [3:0] count;

  input start_stop;

  input load;

  input clock;

  input clear;

  reg [3:0] count;

  always @(posedge clock or posedge clear)

   if (clear) count <= 0;

    else if (load) count <= data;

              else if (start_stop) count <= count + 1;
```

endmodule

2) Για να υλοποιήσουμε έναν μετρητή των 8 δυαδικών ψηφίων, χρησιμοποιούμε δύο μετρητές των 4^{ων} δυαδικών ψηφίων. Πρώτα αρχικοποιούμε τον νέο μας counter με τα σήματα του

```
module counter_8bit(clear,clock,load,start_stop,count,data);

input [7:0] data;

output wire [7:0] count;

input start_stop;

input load;

input clock;

input clear;
```

Στη συνέχεια, φτιάχνουμε τα 2 αντίγραφα του αρχικού μας counter. Ο counter_low αντιπροσωπεύει τα 4 λιγότερα σημαντικά bits του μετρητή, ενώ αντίστοιχα ο counter_high τα 4 πιο σημαντικά bits. Για το λόγο αυτό στον counter_low μπορούν να φορτωθούν στο data τα 4 λιγότερο σημαντικά bits του συνολικού data και αντίστοιχα στον counter_high τα υπόλοιπα 4. Το ίδιο ισχύει και για το count. Κάθε φορά που ο counter_low φτάσει στην τιμή 1111, ο counter_high αυξάνεται κατά 1 και ο counter_low μηδενίζεται. Έτσι, στο start_stop του counter_high ελέγχεται και αν ο άλλος έχει τερματίσει.

```
counter counter_low(.clear(clear), .clock(clock), .load(load),
.start_stop(start_stop),.data(data[3:0]), .count(count[3:0]));

counter counter_high(.clear(clear), .clock(clock), .load(load),
.start_stop(start_stop && (count[3:0] == 4'b1111)),.data(data[7:4]),
.count(count[7:4]));
```

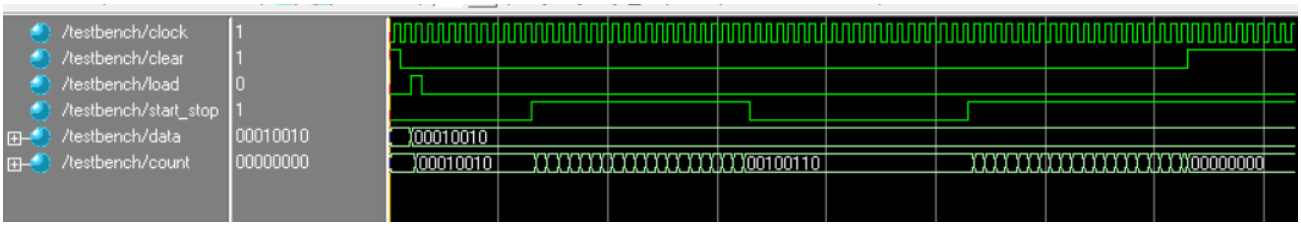
3) Ένα σύνολο διανυσμάτων εξομοίωσης για τον μετρητή είναι το παρακάτω:

```
module testbench;

reg clock;

reg clear;

reg load;

reg start_stop;

reg [7:0] data;
```

```
wire [7:0] count;

counter_8bit uut (
    .clear(clear),
    .clock(clock),
    .load(load),
    .start_stop(start_stop),
    .count(count),
    .data(data)
);

always #5 clock = ~clock;

initial begin
    clock = 0;
    clear = 1;
    load = 0;
    start_stop = 0;
    data = 8'b0;

    #10 clear = 0;
    #10 begin
        load = 1;
        data = 8'd18;
    end
    #10 load = 0;
    #100 start_stop = 1;
```

```
    #200 start_stop = 0;

    #200 start_stop = 1;

    #200 clear = 1;

    #100;

    $finish;

end

initial begin

        $monitor("clear = %b, clock = %b, load = %b, start_stop = %b, data = %d, count = %d",

        clear,clock,load,start_stop,data,count);

end

endmodule
```

4) Εξομοιώνοντας τον σχεδιασμό παίρνουμε



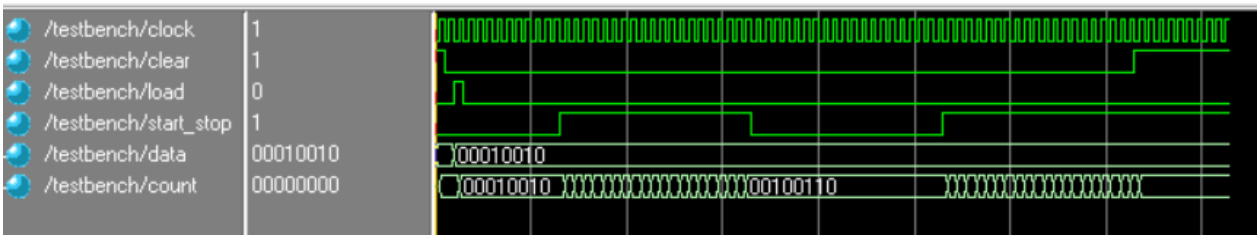
5) Υλοποιούμε τον μετρητή των 4^{ων} δυαδικών ψηφίων με σύγχρονη είσοδο καθαρισμού. Το clear εξαρτάται μόνο από το ρολόι.

```
module counter (clear, clock, load, start_stop, count, data);

  input [3:0] data;

  output [3:0] count;

  input start_stop;

  input load;

  input clock;

  input clear;

  reg [3:0] count;

  always @(posedge clock) begin

   if (clear) count <= 0;
```

```
    else if (load) count <= data;

            else if (start_stop) count <= count + 1;

end

endmodule
```

Ο μετρητής των 8 δυαδικών ψηφίων και το σύνολο διανυσμάτων εξομοίωσης μένουν ίδια.

Τα αποτελέσματα της εξομοίωσης είναι



```
run
# clear = 1, clock = 0, load = 0, start_stop = 0, data =  0, count =  x
# clear = 1, clock = 1, load = 0, start_stop = 0, data =  0, count =  0
# clear = 0, clock = 0, load = 0, start_stop = 0, data =  0, count =  0
# clear = 0, clock = 1, load = 0, start_stop = 0, data =  0, count =  0
# clear = 0, clock = 0, load = 1, start_stop = 0, data = 18, count =  0
# clear = 0, clock = 1, load = 1, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 18
run
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 18
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 18
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 19
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 19
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 20
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 20
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 21
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 21
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 22
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 22
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 23
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 23
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 24
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 24
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 25
run
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 25
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 26
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 26
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 27
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 27
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 28
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 28
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 29
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 29
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 30
```

```
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 32
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 32
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 33
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 33
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 34
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 34
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 35
run
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 35
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 36
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 36
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 37
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 37
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 38
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 38
run
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 38
run
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 0, data = 18, count = 38
# clear = 0, clock = 0, load = 0, start_stop = 0, data = 18, count = 38
```

```
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 38
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 39
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 39
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 40
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 40
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 41
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 41
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 42
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 42
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 43
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 43
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 44
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 44
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 45
run
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 45
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 46
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 46
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 47
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 47
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 48
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 48
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 49
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 49
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 50
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 50
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 51
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 51
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 52
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 52
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 53
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 53
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 54
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 54
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 55
run
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 55
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 56
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 56
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 57
# clear = 0, clock = 0, load = 0, start_stop = 1, data = 18, count = 57
# clear = 0, clock = 1, load = 0, start_stop = 1, data = 18, count = 58
# clear = 1, clock = 0, load = 0, start_stop = 1, data = 18, count = 58
# clear = 1, clock = 1, load = 0, start_stop = 1, data = 18, count =  0
# clear = 1, clock = 0, load = 0, start_stop = 1, data = 18, count =  0
# clear = 1, clock = 1, load = 0, start_stop = 1, data = 18, count =  0
# clear = 1, clock = 0, load = 0, start_stop = 1, data = 18, count =  0
# clear = 1, clock = 1, load = 0, start_stop = 1, data = 18, count =  0
# clear = 1, clock = 0, load = 0, start_stop = 1, data = 18, count =  0
# clear = 1, clock = 1, load = 0, start_stop = 1, data = 18, count =  0
# clear = 1, clock = 0, load = 0, start_stop = 1, data = 18, count =  0
# clear = 1, clock = 1, load = 0, start_stop = 1, data = 18, count =  0
# clear = 1, clock = 0, load = 0, start_stop = 1, data = 18, count =  0
```

6) Εισάγουμε μια καθυστέρηση 20 χρονικών στιγμών στην λειτουργία του μετρητή 4^{ων} δυαδικών ψηφίων. Ο μετρητής των 8 δυαδικών ψηφίων αποτελείται από 2 μετρητές των 4^{ων} δυαδικών ψηφίων, άρα η μέγιστη συχνότητα λειτουργίας του θα είναι $f_{max} = \frac{f_{clk}}{20}$.

Εργασία 6

Α) Θέλουμε ένα κύκλωμα το οποίο θα πολλαπλασιάζει 2 αριθμούς, ο καθένας των 2 δυαδικών ψηφίων και θα απεικονίζει το αποτέλεσμά τους στο 7 segment. αν οι αριθμοί να είναι μη προσημασμένοι . Έστω, a και b οι δύο αριθμοί και s το αποτέλεσμα του πολλαπλασιασμού. Κάνουμε assign στο αποτέλεσμα την αντίστοιχη τιμή του του αποτελέσματος για τις διαφορετικές τιμές των a,b. Το dp είναι μηδέν, αφού a,b μη προσημασμένοι.

a	b	s
01	01	1
01	10	2
01	11	3
10	10	4
10	11	6
11	11	9

Ο κώδικας:

```
module multi1 (a, b, s, dp);
  input [1:0] a, b;
  output [6:0] s;
  output        dp;

  assign s[6:0] = (a == 2'b00 || b == 2'b00) ? 7'b1110111 : //0
                  (a == 2'b01 && b == 2'b01) ? 7'b0010010 : //1
                  (a == 2'b01 && b == 2'b10) ? 7'b1011101 : //2
                      (a == 2'b01 && b == 2'b11) ? 7'b1011011 : //3
                  (a == 2'b10 && b == 2'b10) ? 7'b0111010 : //4
```

```
                (a == 2'b10 && b == 2'b11) ? 7'b1101111 : //6
                7'b1111011;                              //9



  assign dp = 0;
endmodule
```

Β) Θέλουμε ένα κύκλωμα το οποίο θα πολλαπλασιάζει 2 αριθμούς, ο καθένας των 2 δυαδικών ψηφίων και θα απεικονίζει το αποτέλεσμά τους στο 7 segment. αν οι αριθμοί να είναι προσημασμένοι . Κάνουμε assign στο αποτέλεσμα την αντίστοιχη τιμή του του αποτελέσματος για τις διαφορετικές τιμές των a,b. Το dp το ανάβουμε στις περιπτώσεις που το αποτέλεσμα είναι -2 ή -1 για να δηλώσουμε το αρνητικό πρόσημο.

a	b	s	dp
01	01	1	0
01	10	-2	1
01	11	-1	1
10	10	4	0
10	11	2	0

Ο κώδικας:

```
module multi2 (a, b, s, dp);
  input [1:0] a, b;
  output [6:0] s;
  output        dp;


  assign s[6:0] = (a == 2'b00 || b == 2'b00) ? 7'b1110111 : //0
                //(a == 2'b01 && b == 2'b01) ? 7'b0010010 : //1
                (a == 2'b01 && b == 2'b10) ? 7'b1011101 : //-2
                     (a == 2'b01 && b == 2'b11) ? 7'b0010010 : //-1
                (a == 2'b10 && b == 2'b10) ? 7'b0111010 : //4
                (a == 2'b10 && b == 2'b11) ? 7'b1011101 : //2
                7'b0010010;                              //1
```

```
 assign dp = (a == 2'b01 && b == 2'b10) ? 1 : //-2
            (a == 2'b01 && b == 2'b11) ? 1 : //-1
          0;

endmodule
```

Ο χρόνος του χειρότερου μονοπατιού είναι 12.345.

Εργασία 7

Α) Θέλουμε να φτιάξουμε την λειτουργία stop-watch. Το pause παίρνει τιμή από το αν έχει πατηθεί ή όχι ο διακόπτης. Το pause_state είναι ένα flag για τον έλεγχο της κατάστασης του μετρητή. Όταν έχουμε pause, κρατάμε τις τρέχουσες τιμές του μετρητή και τις δίνουμε ως έξοδο. Αντίθετα, όταν κατέβει ο διακόπτης δίνονται ξανά στην έξοδο οι κανονικές τιμές του μετρητή.

Ο κώδικας:

```
if (pause && ~pause_state) begin
            ts_saved <= ts_internal;
            ss_saved <= ss_internal;
            ts <= ts_saved;
            ss <= ss_saved;
            pause_state <= 1;
        end else if (~pause && pause_state) begin
            ts <= ts_internal;
            ss <= ss_internal;
            pause_state <= 0;
        end else if (~pause) begin
            ts <= ts_internal;
            ss <= ss_internal;
```

```
        end
    end
```

Β) Σε αυτό το ερώτημα θέλουμε η μέτρηση των δεκάτων να γίνεται στα led bars. Ακολουθούμε την ίδια λογική με το προηγούμενο ερώτημα και απλά αλλάζουμε την αναπαράσταση των δεκάτων με τον παρακάτω κώδικα :

```
module led_encoder (
  input [2:0] x,
  output reg [9:0] leds
);

  always @(*) begin
    case (x)
        3'd0: leds = 10'b0000000000;
        3'd1: leds = 10'b0000000001;
        3'd2: leds = 10'b0000000011;
        3'd3: leds = 10'b0000000111;
        3'd4: leds = 10'b0000001111;
        3'd5: leds = 10'b0000011111;
        3'd6: leds = 10'b0000111111;
        3'd7: leds = 10'b0001111111;
        3'd8: leds = 10'b0011111111;
        3'd9: leds = 10'b0111111111;
        default: leds = 10'b0000000000;
    endcase
  end
endmodule
```

Εργασία 8

Α) Θέλουμε να φτιάξουμε τη λειτουργία του προηγούμενου πλήκτρου. Όταν ανιχνευθεί ένα scancode πριν δώσουμε στο reg scancode την νέα του τιμή, την κρατάμε στο prev_scancode. Καλούμε την scan_2_7seg και για το prev_scancode και το εμφανίζουμε στο 7 segment.

```
if (f0) // following a scancode of f0. So a key is released !
                begin
                 prev_scancode <= scancode;
                        scancode <= shift[8:1];
                        f0 <= 0;
                end
```